

DPP Assignment 2

Simon Vinding Brodersen(rzn875)

December 1, 2025

1 Flattening with the New, More Efficient Rules

2 Flattening Rule for Scatter inside Map

Let Sx and Si denote the shape of the inner arrays of xss and iss same as S_{arr}^1 in the slides, let Dx denote the flat data of xss , let Di denote the flat data of iss and Dv be the flat data of vss . Then we would have the following flattening rules for the scatter inside a map:

```
let (Bx, _) = mkFlagArray Sx 0 (iota (length Sx))
let (Bi, flags') = mkFlagArray Si (iota (length Si))
let flags = map bool.u32 flags'
let ii1s = sgmScan (+) 0 flags flags'
let glob_idxxs = map2 (\sgm ind -> ind + Bx[sgm]) ii1s Di
in res = scatter Dx glob_idxxs Dv
```

$Bx[i]$ tells us the starting location of the inner list i in Dx . We then find the segment indices for the iss , such that we can add the offset, $Bx[sgm]$, to each of the inner lists of iss . This then gives us the final global index of each of the values. This then means we can directly scatter into Dx with the global indices and given each index corresponds to some value in vss , we don't have to change anything to Dv .

3 Flattening Rule for Histogram inside Map

Much the same as before, let Sx and Si the shape of the inner arrays of $histos$ and iss , same as S_{arr}^1 in the slides, let Dx denote the flat data of $histos$, let Di denote the flat data of iss and Dv be the flat data of vss .

Then we have the following flattening rules for the `reduce_by_index` inside a map:

```
let (Bx, _) = mkFlagArray Sx 0 (iota (length Sx))
let (Bi, flags') = mkFlagArray Si (iota (length Si))
let flags = map bool.u32 flags'
let ii1s = sgmScan (+) 0 flags flags'
let glob_idxxs = map2 (\sgm ind -> ind + Bx[sgm]) ii1s Di
in res = reduce_by_idx (⊙) 0⊙ Dx glob_idxxs Dv
```

The argument for correctness is analogous to that of the previous task.

4 Lifted version of partition2